

PROJECT REPORT

A World in Your Palm

Eight Small Wonders

A complete spherical planet in the browser — eight regions, eight interactive wonders, a real day and night you can walk across, and a small explorer to walk it with. Every tree, cottage, coral, footstep and chime is generated by JavaScript at page load. There is not a single asset file in the project.



SOURCE

7,293 lines

DEPENDENCIES

Three.js only

ASSET FILES

Zero

SHIPS AS

997 KB

DATE

5 Sep 2026

What is in this report

1	Executive summary	10	Build and run — every command
2	The brief	11	Testing and verification
3	Phase 1 — Reverse-engineering the reference	12	Performance engineering
4	Phase 2 — Design decisions	13	Bugs found and fixed
5	Tech stack	14	File reference
6	Architecture	15	How to study this project
7	The engine, system by system	16	Extending the game
8	Call graph — who calls whom	17	Appendix — full API surface
9	Anatomy of one frame		

The eight regions

#	TERRAIN	REGION	THE SMALL WONDER
01	Forest	Cubbon Woods	Light the camp
02	Meadow	Hesaraghatta Fields	Start the windmill
03	Desert	Ramanagara Dunes	Turn the dial
04	Coast	Ulsoor Bay	Ring the bell
05	Reef	Sankey Deep	Wake the reef
06	Volcano	Savandurga Peak	Read the instruments
07	Tundra	Nandi Frost	Ring the chime
08	Hollow	Lalbagh Hollow	Open the jars

Executive summary

What was built, and the one constraint that shaped every decision in it.

A World in Your Palm is a browser game in which a small explorer walks around a complete spherical planet with eight distinct regions, each holding one interactive “small wonder” to wake up.

The defining constraint is that **nothing is loaded**. There is no art pipeline, no `.glb`, no texture atlas, no `.mp3`. Every tree, cottage, coral, character limb, footstep and chime is generated by JavaScript at page load. The entire shippable product is one HTML file you could email to someone.

That constraint drives the architecture. Because props are *functions*, they can be authored as deep hierarchies of hundreds of tiny meshes — the only sane way to write a windmill by hand — and then mechanically flattened before rendering.

The headline result. 6,348 meshes authored in code collapse into 213 merged batches — a 96.6% reduction in draw work with no loss of visual detail. That single function is what makes the approach viable.

Headline numbers

SOURCE

7,293

lines across 11 modules

FUNCTIONS

191

702 comment lines

ASSET FILES

0

everything generated

MESHES AUTHORED

6,348

in JavaScript

MERGED INTO

213

batches — 3.4%

REACHING THE GPU

437

16 instanced

TRIANGLES

269,719

in the scene

NAV GRAPH

4,842

nodes over the globe

ROUTE PLANNING

~5 ms

across the planet

PROPERTY	VALUE
Runtime	Browser, WebGL 2, no server
Language	JavaScript, ES5 style — no transpiler, no bundler
Dependency	Three.js r160 (UMD), vendored — the only one
Regions / wonders / collectibles	8 / 8 / 16
Collision volumes	125 spherical caps
Three.js classes used	54
Self-contained build	997 KB, one file

The brief, and how the reference was read

What was asked, how it was interpreted, and what static analysis recovered from a 916 KB minified file.

2. The brief

“Go through the link like a tester, gamer, senior game developer with 40+ years of experience. Understand the game, don’t leave any single part of it, including the map assets, everything. Now develop exactly the same kind of thing from the observed analysis — just change some small wonder tasks and names, add some extra features, but build it in a similar way, like a pro.”

The reference was a published demo, *Little Planet — A Pocket Adventure*. The instruction was explicitly **not** “copy the file” — it was *understand it, then build your own to the same standard*. That framed the work in three phases.

PHASE	GOAL
1 — Analysis	Dissect the reference down to its algorithms, not its pixels
2 — Design	Re-theme all content; decide what to add beyond the reference
3 — Build	Implement from scratch, verify empirically, ship

Interpretation decisions

- “**Everything, including map assets**” — the reference has no asset files, so “assets” meant *the procedural generators that produce them*. Analysis had to reach the generator level, not the screenshot level.
- “**Change names and some wonder tasks**” — all eight regions renamed and all eight wonders redesigned with new mechanics, not new labels.
- “**Like a pro**” — a real project structure with a build step, a test harness and documentation, rather than one 900 KB file.

3. Reverse-engineering the reference

The reference ships as a single 916 KB HTML file: a minified Three.js bundle plus roughly 133 KB of minified game code. It was analysed statically.

1. Fetch the page source → 916 KB, 4,545 lines
2. Locate `<script>` boundaries → one giant IIFE
3. Split vendor bundle from game code → isolated at offset ~760 KB
4. Index every top-level declaration → ~200 minified symbols mapped
5. Read each subsystem, renaming as understood

Minified identifiers were reconstructed by reading their bodies and inferring intent. For example `jd(s,t,e)` turned out to be a closed-form great-circle-arc versus spherical-cap intersection test.

REFERENCE SYMBOL WHAT IT ACTUALLY IS

<code>Un = 14</code>	Planet radius; <code>Jd</code> is sea level at +0.025
<code>ux(p)</code>	Nearest-biome classification with sinusoidal border noise
<code>Kd(p)</code>	Terrain radius = blended per-biome elevation + detail octaves
<code>Je(root)</code>	The geometry baker — transform folding + colour-to-vertex-colour
<code>jd(a,b,caps)</code>	Closed-form arc-versus-cap visibility test
<code>D3(from,to,...)</code>	A* over a geodesic graph, then string-pulling
<code>Qd class</code>	The spherical walker
<code>vx()</code>	Procedural character rig
<code>Sx()</code>	WebAudio synthesiser with per-biome footstep profiles

The five ideas worth keeping

Analysis concluded that the demo's quality comes from five decisions, all of which were adopted and independently re-implemented.

1. **Props are code, not data.** Removes the entire asset pipeline.
2. **Author deep, then bake.** Deep hierarchies for authoring; flat merged geometry for rendering.
3. **Position is a unit normal, not a point.** The character is glued to a sphere by construction, so "fell through the world" cannot happen.
4. **Obstacles are spherical caps.** Blocking becomes a dot product; visibility becomes one `atan2`.
5. **Synthesise the audio.** Per-material footsteps for free.

Everything else — layout, content, palette, features — was designed fresh.

Design decisions

Eight regions, eight wonders, and six things the reference does not have.

The eight wonders

Each is a toggle that changes the region's state, returns a line of prose, and is replayed on load so the world resumes as you left it.

REGION	ACTION	EFFECT
Cubbon Woods	Light the camp	Campfire, three lamp posts, treehouse string lights, 34 fireflies
Hesaraghatta Fields	Start the windmill	Sails accelerate, 20 bees emerge, 22 flowerbeds bloom
Ramanagara Dunes	Turn the dial	A hidden oasis scales up from nothing — pool, palms, butterfly
Ulsoor Bay	Ring the bell	Bell swings; a sailboat sails a full circuit of the cove and returns
Sankey Deep	Wake the reef	Every coral material lights at once; a 16-fish school appears
Savandurga Peak	Read the instruments	All lava materials brighten and pulse; dish and telescope track
Nandi Frost	Ring the chime	Three aurora curtains unroll across the polar sky
Lalbagh Hollow	Open the jars	Nine jars and every glowing surface rise; 70 motes swarm

Six additions beyond the reference

ADDITION	WHY
An eighth region	The reference has seven; a night-themed region justified the bloom pipeline
Real day/night cycle	A directional sun on a tilted orbit — half the globe is genuinely dark and you can <i>walk</i> across the terminator
Carried lantern with a point light	Auto-lights at dusk and casts an actual pool of light — makes the night side worth visiting rather than merely survivable
Companion creature	Follows with its own spherical walker and pathfinding, so it never clips through trees
16 collectible wisps	A secondary exploration loop independent of the wonders
Compass and photo mode	Wayfinding, and a PNG export for sharing

A decision worth defending. The held sky settings — day, dusk, night — do not pin a clock time. They solve for the sun position that produces that sky *at the explorer's current location*, and keep re-solving as you walk. On a planet you can circumnavigate, "night" pinned to a clock would mean broad daylight wherever you happened to be standing.

Tech stack and architecture

One dependency, eleven modules, an acyclic dependency graph, and a deliberate refusal to add a bundler.

5. Tech stack

LAYER	CHOICE	WHY
Rendering	Three.js r160 (UMD)	The only dependency. UMD, not ESM, so classic <code><script></code> tags work from <code>file://</code> with no server and no CORS
Graphics API	WebGL 2	Needed for multisampled half-float render targets
Language	ES5-style JS	<code>var</code> , <code>function</code> , IIFE modules. No build step at runtime
Audio	Web Audio API	Oscillators, buffers, biquads, convolver — all synthesised
Persistence	localStorage	Key <code>tiny-world-save-v1</code>
Input	Pointer Events	One code path for mouse, touch and pen
UI	Plain HTML + CSS	<code><dialog></code> for panels; no framework

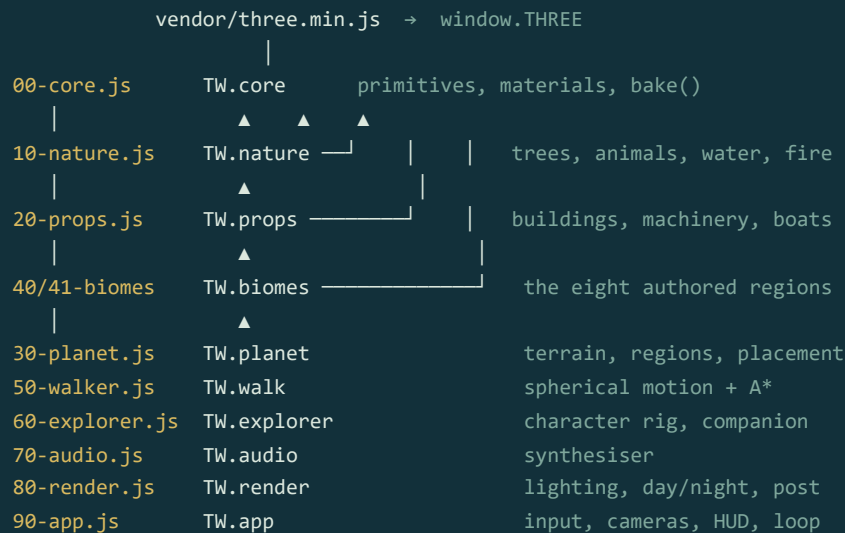
Three.js surface actually used — 54 classes

Scene	Group	Object3D	Mesh	Points	InstancedMesh	PerspectiveCamera	OrthographicCamera	Raycaster
Color	Vector2	Vector3	Quaternion	Matrix4	MathUtils	BufferGeometry	BufferAttribute	
Float32BufferAttribute	InstancedBufferAttribute	BoxGeometry	IcosahedronGeometry	OctahedronGeometry				
SphereGeometry	CylinderGeometry	ConeGeometry	TorusGeometry	CircleGeometry	RingGeometry	TubeGeometry		
ExtrudeGeometry	ShapeGeometry	Shape	CatmullRomCurve3	MeshStandardMaterial	MeshBasicMaterial			
ShaderMaterial	PointsMaterial	DataTexture	AmbientLight	HemisphereLight	DirectionalLight	PointLight		
WebGLRenderer	WebGLRenderTarget	HalfFloatType						

Notably **no Three.js addons**. `EffectComposer` and `BufferGeometryUtils` are ESM-only and would have forced a bundler. Geometry merging and the entire post-processing chain were written by hand instead — about 150 lines in total.

6. Architecture

Eleven files load in numeric order as classic scripts. Each is an IIFE that attaches one namespace to a global `TW` object. Load order is the dependency order — a module can only use what is already defined above it.



Module map. Dependency edges below were extracted from the source, not asserted.

MODULE	DEPENDS ON
00-core.js	— (only Three.js)
10-nature.js	00-core.js
20-props.js	00-core.js, 10-nature.js
40/41-biomes	00-core.js, 10-nature.js, 20-props.js
30-planet.js	00-core.js, 10-nature.js, 40/41-biomes
50-walker.js	— (only Three.js)
60-explorer.js	00-core.js, 50-walker.js
70-audio.js	— (only Web Audio)
80-render.js	00-core.js
90-app.js	00-core.js, 30-planet.js, 50-walker.js, 80-render.js, 60-explorer.js, 70-audio.js

The graph is acyclic with one deliberate inversion: `30-planet.js` calls `40/41-biomes.buildAll(ctx)` and hands it a context object, while the biome files call back into `ctx.place()`, `ctx.path()` and `ctx.wonder()`. **Content does not know about spheres; the planet does not know about content.**

Why no bundler. Classic script tags load from `file://` without a server, without CORS problems, and without a toolchain. The cost is manual load ordering and no tree-shaking; the benefit is that a contributor can edit a file, press reload, and see the change. `build.mjs` recovers the single-file distribution when one is wanted.

The engine, system by system

The primitive kit, world generation, spherical motion, navigation, the character, rendering and audio.

7.1 The primitive kit and the baker

Six primitives build the entire world.

FUNCTION	PRODUCES
<code>box(p, x,y,z, w,h,d, col)</code>	Axis-aligned box
<code>rock(p, ..., coarse)</code>	Randomly tumbled icosahedron — every boulder, bush and blob
<code>cone(p, ..., rBot, h, seg, rTop)</code>	Tapered cylinder; <code>rTop = 0</code> gives a cone
<code>tube(...)</code>	<code>cone</code> with equal radii
<code>link(p, a, b, r, col)</code>	A cylinder <i>between two points</i> — ropes, branches, struts
<code>poly(verts, faces, colors)</code>	Raw triangle soup for anything bespoke

bake() — the load-bearing function

For every static, opaque, single-material mesh under a root:

1. Fold the world transform into the vertex positions.
2. Fold the flat material colour into a **vertex-colour attribute**, and swap in a shared white `vertexColors` material. *This is the trick* — it lets meshes of different colours merge into one geometry.
3. Concatenate into one buffer per `(material, castShadow, receiveShadow)` bucket.

Anything marked `userData.dynamic` — and its whole subtree — is skipped, so animated joints keep their own transforms.

AUTHORED

6,348

meshes

MERGED INTO

213

batches

GROUPS PRUNED

2,217

empty husks

Determinism. `rand(a,b)` is a seeded LCG. Nothing in generation may call `Math.random()`, so the planet is byte-identical on every machine and in every screenshot.

7.2 Generating a world

Eight biome centres are placed by latitude and longitude. Any point on the sphere belongs to the nearest centre — but the distance metric is perturbed:

```
score = dot(centre, p) + 0.018 * sin(p.x*9 + i*2.3) * sin(p.y*8 - p.z*6 + i*0.6)
```

Without the noise term the borders would be clean great-circle arcs and look manufactured. `classify()` also returns the runner-up and the margin, which lets elevation blend across a border instead of forming a cliff.

The terrain mesh is `IcosahedronGeometry(14, 40)` — 100,860 vertices. Each face is displaced, classified, and given a per-face colour with a small random tint. Flat-shaded, so the facets *are* the aesthetic.

The placement context

Biome files never touch spheres. They receive:

API	PURPOSE
<code>ctx.place(id, x, z, build, opts)</code>	Drop a prop at local metres; handles the tangent frame
<code>ctx.path(id, points, width, color)</code>	Drape a footpath over the terrain
<code>ctx.block(id, x, z, radius)</code>	Register a collision cap
<code>ctx.wonder(spec)</code>	Register an interactive landmark
<code>ctx.wisp(id, x, z)</code>	Register a collectible
<code>ctx.sky(build)</code>	A group whose local frame is world space (the aurora)
<code>ctx.tick(fn)</code>	Per-frame callback outside the prop hierarchy

7.3 Walking on a sphere

The explorer has **no position**. It has a unit `normal` — the direction from the planet centre to its feet — and a unit tangent `forward`. Moving is rotating the normal along a great circle:

```
advance(from, dir, distance, radius):
    t = tangent(dir, from)
    a = distance / radius
    return from*cos(a) + t*sin(a)    // normalised
```

Height comes from `surfaceRadius(normal)`, so the character is welded to the terrain by construction. There is no gravity solve and no ground raycast. After each move the facing vectors are **parallel-transported** along the step with a quaternion built from the before/after normals, so they stay tangent without drift.

Obstacles are spherical caps. Being inside one is `dot(p, cap.normal) > cos(angularRadius)`. `resolve()` pushes a point back out along the tangent, three iterations, every frame.

7.4 Navigation

Along the great circle from a to b , the dot product with a cap centre is:

$$f(t) = h \cdot \cos t + d \cdot \sin t = R \cdot \cos(t - \phi) \quad \text{where } R = \text{hypot}(h,d), \phi = \text{atan2}(d,h)$$

So the closest approach to any obstacle is **one `atan2`** — no marching, no sampling. This is what makes planet-scale A* cheap enough to run inside a click handler.

STAGE	DETAIL
Graph	<code>IcosahedronGeometry(1, 21)</code> deduplicated to 4,842 nodes . Nodes inside an obstacle are closed; edges survive only if the arc between them is clear
Caching	Built once, cached on the collider array via a <code>WeakMap</code> , and pre-warmed at load so the first click never pays the ~76 ms cost
Search	A* with a great-circle heuristic and a binary heap, using a virtual goal node so multiple entry portals relax cheaply
Smoothing	String-pulling : from each anchor, skip to the furthest waypoint still in clear line of sight. A staircase becomes 2–5 long natural legs

Two-tier design, verified. If A* returns `null` — for instance if the walker is standing inside a cap — the walker falls back to reactive steering: head toward the target, and when an obstacle is in the way, commit to one side and slide. Measured: from a blocked start with `_route === null`, the walker escaped the cap and covered **10.9 m in 2 simulated seconds**.

7.5 The character

A real skeleton of nested pivots — no skinning and no animation data.

```
root
├─ body                whole-body bob, lean, roll
│  ├─ torso            coat, pockets, straps, chest compass
│  │  └─ pack          bedroll, mug, map, buckles
│  ├─ legL / legR
│  │  └─ thigh → knee → ankle
│  ├─ armL / armR
│  │  └─ shoulder → elbow → hand
│  │     └─ lantern    left hand – swings, lights at dusk
│  └─ head
│     ├─ eyes x2       blink on a 5.4 s cycle
│     ├─ hat           brim, seams, goggles on the crown
│     └─ scarf → tail  trails with movement
```

The walk cycle is a handful of sines driven by **distance travelled**, not wall time — so the gait always matches the speed. Five blend weights chase their targets exponentially and layer poses on top of each other. Each joint is baked individually with `keepRootLive`, so a joint’s own static meshes merge while its children stay independently animatable.

The companion owns a **full walker** — it pathfinds around obstacles exactly as the player does, follows when the gap exceeds 2.4 m, and sits down when it closes to 1.3 m.

7.6 Lighting and the post chain

The sun runs on a clean tilted great circle, $\text{sunDir}(t) = A \cdot \cos 2\pi t + B \cdot \sin 2\pi t$. Keeping it analytic is what makes the held sky settings work: since $\text{dot}(n, \text{sunDir}(t)) = R \cdot \cos(2\pi t - \phi)$, the time that produces a given sun elevation at a given point is **one** `acos`.

```
solveTimeFor(normal, target):  
  a = normal.A ; b = normal.B  
  R = hypot(a, b) ;  $\phi = \text{atan2}(b, a)$   
  return ( $\phi + \text{acos}(\text{clamp}(\text{target} / R, -1, 1))$ ) /  $2\pi$ 
```

A full day is 420 s. Local daylight is $\text{clamp}(\text{dot}(\text{normal}, \text{sunDir}) \cdot 2.6 + 0.42, 0, 1)$ and drives ambient level and colour, hemisphere light, moon fill, star opacity, atmosphere tint, the night colour grade, and the lantern.

```
scene → HDR half-float target (4× MSAA)  
  |  
  ↳ bright pass (soft-knee threshold) → ½-res  
    ↳ blur H → blur V → blur H(×2.4) → blur V(×2.4)  
  ↳ composite: scene + bloom  
                × exposure  
                night colour grade  
                Khronos PBR Neutral tone map  
                vignette  
                sRGB encode  
                premultiplied alpha → screen
```

The post-processing chain — hand-written, no Three.js addons.

Explorer occlusion. Props between the camera and the explorer dither away using a fixed 4×4 Bayer pattern, injected into every opaque standard material via `onBeforeCompile`. Fixed rather than noise — a noise pattern sparkles when you stand still.

Graceful degradation. Without WebGL 2 the whole chain is replaced by a direct render with ACES tone mapping. Below 40 fps the pixel ratio drops; below that again, bloom and soft shadows are dropped.

7.7 The synthesiser

No audio files. Voices are built per event and self-dispose via `onended`.

SOUND	CONSTRUCTION
Footstep	Band-limited noise burst (lowpass sweep + highpass) + low sine "thud"
Splash	Longer, brighter burst with a higher highpass
Wind	One looping brown-noise buffer through a lowpass whose corner follows the region
Discovery	Pentatonic tone, one note per region — nothing can sound wrong
Room	Synthetic impulse response through a <code>ConvolverNode</code>

Per-region ground profiles tune filter cutoff, duration, level and thud pitch — sand is long and bright (2600 Hz, 145 ms), volcanic rock is short and hard (3400 Hz, 45 ms).

Call graph and the anatomy of one frame

What calls what, once at load and then sixty times a second.

8.1 Build-time flow — once, at load

```

90-app.js
├─ TW.render.createLighting(scene)           sun, moon, ambient, hemisphere
├─ TW.planet.build({terrainDetail:40, scatter:1650})
│   ├─ IcosahedronGeometry(14,40) → displace → classify → vertex colours
│   ├─ sea shell (IcosahedronGeometry(14.025, 34))
│   └─ TW.biomes.buildAll(ctx) ────────── eight region builders
│       │   └─ ctx.place(id,x,z,build) ← each calling...
│       │   └─ ctx.path / block / wonder
│       └─ build(g) calls:
│           └─ TW.nature.pine/palm/...   ← 10-nature.js
│           └─ TW.props.cottage/well/... ← 20-props.js
│               └─ TW.core.box/rock/cone/link/poly
├─ trails between regions · 1,650-point Fibonacci scatter
├─ clouds (1 InstancedMesh) · wonder markers · wisps
├─ TW.core.bake(propRoot)           6,348 meshes → 213 batches
├─ pruneEmpty(propRoot)             2,217 groups removed
├─ TW.planet.buildSky(scene)         stars + atmosphere rim shader
├─ TW.render.installOcclusion(...)    patches every opaque material
├─ TW.explorer.create()              rig, each joint baked
├─ new TW.walk.Walker({...})
├─ TW.explorer.createCompanion({...}) owns its own Walker
├─ TW.render.createPost(...)         HDR target + bloom + composite
├─ lighting.startAt(spawn, 0.66)    open in daylight where we stand
└─ TW.walk.findRoute(...)            pre-warm the 4,842-node graph
  
```

8.2 Per-frame flow

```
frame(now)
├─ camera.matrixWorld → cameraRight / cameraUp → TW.walk.tangent()
├─ walker.update(dt, {x, y, forward, right, running})
│   ├── _steer()          only when auto-walking; route or reactive dodge
│   ├── advance()         great-circle step
│   ├── resolve()         push out of collision caps
│   └─ parallel-transport the forward vectors
├─ explorer.update(dt, {time, speed, swimming, running, airborne, interacting})
│   └─ ~40 joint rotations from blended sines
├─ companion.update(dt, walker.normal, time)  its own Walker
├─ lighting.update(dt, walker.normal)
│   ├── auto: advance timeOfDay | held: solveTimeFor(normal, target)
│   └─ ambient / hemi / moon / sun colour from local daylight
├─ bodies.update(sunDir, camera) · post uniforms ← night, bloom
├─ explorer.setLanternLit(night > 0.42) + point-light intensity
├─ updateCamera(dt) · updateHud() · checkWisps()
├─ TW.core.tickAll(animTime)      86 registered animation callbacks
├─ world.update(animTime)         markers, wisps, clouds, wonder state
├─ confetti integration · audio.update(dt, {speed, swimming, biome, night})
├─ renderer.shadowMap.needsUpdate on a time budget, not every frame
├─ post.render()                  scene → bloom → composite → screen
├─ adaptive quality check         wall-clock fps, two downgrade steps
└─ telemetry publish              every 0.25 s of simulation time
```

9. Anatomy of one frame

#STEP	NOTES
1requestAnimationFrame	<code>dt</code> clamped to 50 ms so a stall cannot teleport the player
2Bail if hidden	A background tab does nothing
3Build tangent basis	Camera axes projected onto the explorer's tangent plane
4Read input	Keys, latched taps, joystick — zeroed while a dialog is open
5walker.update()	Steer → advance → resolve → transport → turn → gravity
6Place explorer	<code>normal × (surfaceRadius + height)</code>
7Animate	Character joints, companion, occlusion uniform
8Lighting	Sun position, local daylight, all derived colours
9Camera	Lerp position, look-at and up vector
10HUD	Nearby wonder, region change, compass bearing
11World ticks	86 animation callbacks + world updaters
12Audio	Footstep accumulator, wind filter targets
13Shadows	Budgeted: 45 ms moving, 180 ms idle
14Render	Scene → HDR target → 5 bloom passes → composite
15Adaptive quality	Wall-clock fps, two downgrade steps
16Telemetry	JSON state for the test harness

Build and run — every command

All commands run from the project root in the VS Code terminal (PowerShell).

Just play it

No build, no server, no install.

```
# Windows – open in the default browser
start index.html
```

Or paste `file:///D:/GAMES/tiny_world/index.html` into any browser, or double-click `index.html`.

Local server — live reload, and phone testing

```
live-server          # → http://127.0.0.1:8080, reloads on save
live-server --port=3000 # different port
live-server --no-browser # don't open a tab
```

For a phone on the same Wi-Fi:

```
live-server --host=0.0.0.0 --port=8080
ipconfig | Select-String IPv4      # find your PC's address
# then open http://<that-IP>:8080 on the phone
```

Allow the Windows Firewall prompt on **Private** networks. If `Live-server` is not on `PATH`, `npx Live-server` works identically.

Build the single file

```
node build.mjs
```

Inlines `vendor/three.min.js` and all 11 sources into `dist/tiny-world.html` — 997 KB, works offline from a USB stick. Run it after editing any source.

Verification harness

```
# one-time install
cd tools
npm install
cd ..

# then
node tools/smoke.mjs           # loads and renders?      ~30 s
node tools/smoke.mjs dist/tiny-world.html # same, against the build
node tools/metrics.mjs         # the numbers in this report
node tools/playthrough.mjs     # full acceptance run    ~5 min
```

Every script exits non-zero on failure, so they drop straight into CI. They need Chrome or Edge already installed — set `CHROME_PATH` if it is somewhere unusual.

Regenerate this report

```
node tools/make-pdf.mjs      # docs/report.html → the PDF you are reading
```

Command reference

COMMAND	WHERE	PURPOSE
<code>start index.html</code>	project root	Play
<code>live-server</code>	project root	Dev server with live reload
<code>live-server --host=0.0.0.0</code>	project root	Expose to phones on the LAN
<code>node build.mjs</code>	project root	Rebuild <code>dist/tiny-world.html</code>
<code>npm install</code>	<code>tools/</code>	Install the harness (once)
<code>node tools/smoke.mjs</code>	project root	Fast health check
<code>node tools/metrics.mjs</code>	project root	Measure the world
<code>node tools/playthrough.mjs</code>	project root	Full acceptance run
<code>node tools/make-pdf.mjs</code>	project root	Rebuild this PDF
<code>git add -A ; git commit ; git push</code>	project root	Publish

Testing and performance

How a graphical, stateful game is verified without mocks — and what it costs to draw.

11. Testing and verification

The game is verified by **driving the real build in a real browser**. Two deliberate hooks make that possible: an `<output id="telemetry">` element carrying full runtime state as JSON, and a `TW.app` namespace with live handles to the renderer, scene, walker, world and lighting.

A lesson worth recording. The harness first read *everything* from telemetry and produced false failures. Telemetry refreshes on the **simulation** clock, which crawls under software rasterisation. Anything time-sensitive must read live state through `TW.app`; telemetry is for the human-facing summary and for checking that the channel itself works.

CHECK	METHOD
Loads and renders	Wait for <code>canvas.dataset.ready</code>
No errors	<code>pageerror</code> , <code>console.error</code> , <code>requestfailed</code> all fail the run
All 8 wonders	Teleport, <code>interact()</code> , assert the found count
All 8 regions	Assert <code>regions === 8</code>
Sky resolves locally	Poll until daylight settles; assert <code>day > 0.9</code> , <code>night < 0.1</code>
Swimming	Assert <code>walker.swimming</code> in the reef
Cross-planet navigation	<code>findRoute()</code> across the globe
Auto-walk arrival	Poll until <code>walker.target</code> clears
Save / restore	Reload, assert 8 wonders return
Spawn in daylight	Assert <code>daylight ≥ 0.5</code> after reload
Mobile layout	Emulate iPhone; assert no overflow and no overlaps
Touch joystick	Synthetic touch drag; assert distance walked

Latest result — both targets pass

PLAYTHROUGH – index.html		PLAYTHROUGH – dist/tiny-world.html	
wonders activated	8 / 8	wonders activated	8 / 8
regions visited	8 / 8	regions visited	8 / 8
sky: day / dusk	1.00 / 0.495	sky: day / dusk	1.00 / 0.547
sky: night	0.00	sky: night	0.00
swims in the reef	true	swims in the reef	true
cross-planet route	4 wp, 5.2 ms	cross-planet route	4 wp, 4.6 ms
auto-walk arrives	true	auto-walk arrives	true
photo mode	true	photo mode	true
save survives	8 wonders	save survives	8 wonders
problems: none		problems: none	

Known limitation. Headless Chrome falls back to SwiftShader, a software rasteriser, so the harness runs at roughly 2–3 fps. That is fine for correctness and says *nothing* about real frame rate. The adaptive-quality ladder was verified to trigger, but performance on real hardware has not been measured here and should be checked on target devices.

12. Performance engineering

MEASURE	BEFORE	AFTERHOW
Meshes in scene	926	437Instancing + sub-part baking
Draw calls (follow view)	438	~200Same
Scene-graph groups	1,527	353 <code>pruneEmpty()</code> after baking

- **Smoke and flames** → `InstancedMesh`. Puffs now fade by shrinking rather than by per-instance alpha, which instancing cannot express.
- **Animals baked per part**. One critter went from ~8 meshes to 2.
- **Kelp rebuilt as a single swept ribbon** instead of a five-segment stack: 26 fronds, 26 meshes.
- **Clouds** → one `InstancedMesh` for all 52 lumps.
- **Flower beds and the oasis baked**, saving ~150 meshes in one region.

RUNTIME TECHNIQUE	EFFECT
Shadow map on a time budget	Re-rendered every 45 ms moving, 180 ms idle — not every frame
Bloom at half resolution	5 fullscreen passes at ¼ the pixel cost
Pixel-ratio cap	<code>min(dpr, sqrt(2.3M / (w·h)))</code> — targets a pixel budget, not a ratio
Navigation graph cached	<code>WeakMap</code> on the collider array; built once, pre-warmed
Adaptive quality	Two downgrade steps below 40 fps on the wall clock

Bugs found and fixed

Every one was caught by automated testing or by inspecting rendered output.
None were found by reading the code.

#BUG	SYMPTOM	ROOT CAUSE	FIX
1fps counter lied	Always reported ~20 fps	Averaged the <i>clamped</i> <code>dt</code> (max 50 ms), so it could never report below 20	Measure against <code>performance.now()</code>
2“Night” showed daylight	Holding night left the sky bright	Phase pinned a clock time, not a local sun elevation	Solve <code>solveTimeFor()</code> analytically each frame
3Black background	CSS page gradient never visible	Composite wrote opaque alpha over the whole canvas	Carry scene coverage into premultiplied output alpha
4Could open at night	Spawn sometimes in darkness	<code>timeOfDay</code> initialised to a constant	<code>startAt(spawn, 0.66)</code> solves for morning at spawn
5Lava floated	Flows hovered at the far end	One prop anchor spanning 6 m of a 20° slope	Rebuilt with <code>ctx.path()</code> , which drapes over terrain
6Crater props buried	Rim rocks sank into the ground	Same single-anchor problem inside the crater	Flattened the crater floor in the elevation function
7Aurora flew off	Flat sheets floating in space	Built in a prop’s local frame, ignoring curvature	Rebuilt in world space via <code>ctx.sky()</code>
8First click stuttered	~76 ms hitch on the first auto-walk	Navigation graph built lazily	Pre-warm at load
9Sun drew through the planet	Sky discs visible through terrain	<code>depthTest: false</code> on the billboards	Enabled depth testing
10Harness false negatives	Tests failed on working features	Read telemetry, which runs on the simulation clock	Read live state via <code>TW.app</code>

File reference

Where everything lives, and how big it is.

FILE	LINES	FNS	COMMENTS	CONTENTS
index.html	395	—	—	Page, interface, all CSS, responsive and touch layout
src/00-core.js	401	24	16%	Primitives, materials, RNG, <code>bake()</code> , merging
src/10-nature.js	572	23	7%	Trees, plants, rocks, water, fire, motes, animals
src/20-props.js	569	26	7%	Shelters, fences, lanterns, boats, bridges, machinery
src/30-planet.js	767	22	11%	Terrain, regions, placement context, scatter, sky
src/40-biomes-a.js	813	5	11%	Cubbon Woods, Hesaraghatta Fields, Ramanagara Dunes, Ulsoor Bay
src/41-biomes-b.js	890	10	11%	Sankey Deep, Savandurga Peak, Nandi Frost, Lalbagh Hollow
src/50-walker.js	592	12	12%	Spherical motion, caps, A*, string-pulling
src/60-explorer.js	592	13	9%	Character rig, animation, companion, waypoint marker
src/70-audio.js	433	19	6%	Web Audio synthesiser
src/80-render.js	611	15	12%	Lighting, day/night, bloom, tone mapping, occlusion
src/90-app.js	1,053	22	5%	Renderer, cameras, input, HUD, persistence, frame loop
build.mjs	54	—	—	Single-file bundler
tools/	~350	—	—	Headless verification harness
Total	7,890	191		

```

tiny_world/
├─ index.html          ← play this
├─ README.md  REPORT.md  build.mjs
├─ vendor/three.min.js  r160 UMD, vendored unmodified
├─ src/                 the 11 modules above
├─ tools/               smoke · metrics · playthrough
├─ docs/               this report + make-pdf.mjs
└─ dist/tiny-world.html generated — 997 KB, self-contained

```

Constants worth knowing

CONSTANT	VALUE
Planet radius	14
Sea level	14.025
Terrain detail	40
Scatter points	1,650
Walk / run / swim	2.6 / 4.7 / 2.9 m·s ⁻¹

CONSTANT	VALUE
Jump / gravity	5.6 / 15
Body / plan clearance	0.26 / 0.39 m
Nav graph detail	21 → 4,842 nodes
Day length	420 s
Save key	tiny-world-save-v1

Studying and extending the project

A reading order that builds understanding, and the rules for adding to it.

15. How to study this project

Session 1 — the idea 1 hour

1. **Play it.** Find all eight wonders. Press **N** at Labyrinth Hollow.
2. Read `src/00-core.js` top to bottom. It is the smallest file and everything else is built from it.
3. In the browser console: `TW.core.bake.totalMerged` → 6348, `TW.core.bake.totalBatches` → 213.

Session 2 — how a world gets made 2 hours

4. Read `TW.nature.pine` and `TW.props.cottage`. They are *just functions returning groups*.
5. Read `ctx` in `src/30-planet.js` — the boundary between engine and content.
6. Read one region builder in `src/40-biomes-a.js`. It reads like a planting list.
7. Change a colour, save, reload. That loop is the whole point of the no-bundler decision.

Session 3 — the mathematics 2 hours

8. Read `src/50-walker.js`. Take `arcIsClear()` slowly — write out $f(t) = R \cdot \cos(t - \phi)$ on paper.
9. Plan a route from the console and inspect the waypoints.
10. Read `solveTimeFor()` in `src/80-render.js` — the same trick applied to sunlight.

Session 4 — presentation 2 hours

11. Read `createExplorer()`. Set `moveBlend` by hand and watch the pose change.
12. Read `createPost()` and follow one pixel through the chain.
13. Read `frame()` in `src/90-app.js` — the whole game in one function.

Live experiments

```
TW.app.lighting.setPhase('night'); // hold the sky
TW.app.walker.walkSpeed = 8;      // sprint
TW.app.post.bloomEnabled = false; // A/B the bloom
TW.app.travelTo(TW.planet.normalAt('cinderpeak', 0, 3));
TW.app.scene.traverse(o => { if (o.isMesh) o.material.wireframe = true; });
```

Internal IDs remain the original development names — `mooswood`, `cinderpeak` — because they are the keys existing save files were written with. Only the display names changed.

16. Extending the game

Add a prop

```
// in src/20-props.js
P.watchtower = function (parent, x, z, height) {
  var g = C.group(parent, x, 0, z);
  for (var s = -1; s <= 1; s += 2)
    for (var t = -1; t <= 1; t += 2)
      C.link(g, [s*0.4, 0, t*0.4], [s*0.22, height, t*0.22], 0.05, PAL.wood);
  C.box(g, 0, height, 0, 0.7, 0.06, 0.7, PAL.wood);
  P.gableRoof(g, 0.8, 0.8, height + 0.5, height + 0.9, PAL.roofRed);
  return g;
};
```

It bakes automatically. Use it with `ctx.place('mosswood', 2, -1, g => P.watchtower(g, 0, 0, 1.8))`.

Add a region

1. Append an entry to `BIOMES` in `src/30-planet.js` — pick a lat/lon well clear of the others.
2. Add region-specific shaping in `regionElevation()` if wanted.
3. Push a builder onto `B.list` in a biome file.
4. Add a footstep profile to `GROUND` in `src/70-audio.js`.

Rules to respect

RULE	WHY
Never call <code>Math.random()</code> during generation	Use <code>ctx.rand()</code> — the world must be deterministic
Anything animated needs <code>C.dyn()</code> or <code>C.pivot()</code>	Otherwise <code>bake()</code> will merge it and freeze it
Long props need multiple anchors, or <code>ctx.path()</code>	One anchor over a slope hovers at the far end (bugs 5 and 6)
Register a collider for anything solid	<code>ctx.block(id, x, z, radius)</code>
Run <code>node build.mjs</code> after editing	Otherwise <code>dist/</code> goes stale

Full API surface

Everything each module exposes.

TW.core 33 exports

TAU DEG clamp lerp smoothstep damp rand reseed makeRandom mat dyn tickAll tickers
group pivot mesh box rock cone tube torus sphere link beam poly slab ribbon
mergeGeometries bake countTriangles UNIT_BOX UNIT_ICO0 UNIT_ICO1

TW.nature 23 exports

pine broadleaf birch palm snag mushroom grass flower bush berryBush reeds boulders
crystals signpost pond waterfall flames smoke motes bird fish butterfly critter

TW.props 26 exports

gableRoof window door cottage tent fence lamppost stringlights crate barrel basket sack
bench woodpile sailboat jetty ropeBridge windmill pump cart telescope mast campfire bell
well steppingStones

TW.planet 15 exports

RADIUS SEA UP BIOMES BY_ID latLon normalAt localAt biomeAt classify terrainRadius
surfaceRadius isWater build buildSky

TW.walk 10 exports

Walker arcLength tangent advance resolve arcIsClear buildCaps findRoute BODY_CLEARANCE
PLAN_CLEARANCE

TW.explorer · TW.render · TW.audio · TW.biomes

SKIN create createWaypointMarker createCompanion createPost createLighting createSunAndMoon
installOcclusion DAY_LENGTH PHASE_ORDER create NOTES GROUND list buildAll

TW.app debug and test surface

renderer scene camera post lighting world walker explorer companion audio travelTo
interact setView goHome save view found collected

All figures in this report were measured from the running build via `node tools/metrics.mjs` and `node tools/playthrough.mjs` .
Regenerate this document with `node tools/make-pdf.mjs` .